Exercise – Understanding convergence concepts using simulation

1. Aim of the exercise

This exercise illustrates convergence concepts that underlie many econometric derivations. Using simulation, we study how estimation error evolves as the sample size increases and how this behavior can be formalized through stochastic convergence, small-o_p notation, and Big-$\mathcal{O}_p$ notation.

2. Theoretical background

A central question in econometrics is how estimation error behaves as the sample size increases. The key tools used to study this question are deterministic convergence, stochastic convergence, the Law of Large Numbers (LLN), and the Central Limit Theorem (CLT). These concepts provide the theoretical foundation for the simulations and convergence results developed in this exercise. Readers are advised to first review the dedicated LLN and CLT exercises, where these results are introduced and discussed in greater detail.

3. Average absolute error and convergence rate

In the dedicated exercise on the Law of Large Numbers, we used a coin-toss experiment to demonstrate how the sample mean approaches the population mean as the sample size increases. In this exercise, we return to the same example and use it as a running illustration of the convergence concepts introduced below.

To investigate how rapidly the sample mean converges to the population mean, we calculate the average absolute error across the 1,000 Monte Carlo replications,

$$\frac{1}{1000}\sum_{r=1}^{1000}\left|\bar{x}_{n,r}-\mu\right|.$$

Figure 1 plots this quantity. The figure is produced by simulating 1,000 samples for each sample size ranging from 1 to 991 coin tosses in increments of 10, calculating the sample proportion in each replication, and then computing the average absolute deviation of the sample proportion from the true probability of 0.5.

To assess the convergence rate, the empirical error path is compared with the candidate decay curves $1/\sqrt{n}$ and $1/n$. The figure shows that the average absolute error closely follows the $1/\sqrt{n}$ benchmark rather than the faster $1/n$ benchmark. This suggests that the estimation error decreases approximately in proportion to $1/\sqrt{n}$ rather than the faster rate $1/n$. In econometrics, this behavior is commonly referred to as convergence at the root-n rate. As we will see in Section 6, this square-root decay is the direct mathematical footprint of the CLT.

```
15 %% 3. Set the parameters of the simulation
16
17 % 3.1. Define alternative sample sizes (number of coin tosses)
18 N_tosses = 1:10:991;
19
20 % 3.2. Define the number of alternative sample sizes
21 N_samples = length(N_tosses);
22
```

```
% 3.3. Define the number of simulated samples
N_sim = 1000;

%% 4. Define the population parameters

% 4.1. Define the mean for a fair coin
mean_population = 0.5;

% 4.2. Define the variance for a fair coin
var_population = 0.25;

%% 5. Generate data and calculate the sample mean

% 5.1. Preallocate memory for the simulated sample means
means_samples = NaN(N_sim,N_samples);

% 5.2. Simulate coin tosses and calculate the sample mean
for j = 1:N_samples
    % Generate N_sim independent samples of size N_tosses(j)
    sequence_of_tosses = random('Binomial',1,0.5,[N_tosses(j),N_sim]);
    % Calculate the sample mean for each simulated sample
    means_samples(:,j) = mean(sequence_of_tosses,1)';
end

%% 6. Visualizing the convergence rate

% 6.1. Calculate the average absolute distance at each sample size
mean_abs_error = mean(abs(means_samples - mean_population),1);

% 6.2. Compute the scale factor for the candidate 1/sqrt(n) decay curve
root_n_curve_scale = sqrt(N_tosses(1)) * mean_abs_error(1);

% 6.3. Generate the candidate 1/sqrt(n) decay curve
root_n_decay_curve = root_n_curve_scale ./ sqrt(N_tosses);

% 6.4. Compute the scale factor for the candidate 1/n decay curve
one_over_n_curve_scale = N_tosses(1) * mean_abs_error(1);

% 6.5. Generate the candidate 1/n decay curve
one_over_n_decay_curve = one_over_n_curve_scale ./ N_tosses;

% 6.6. Plot the empirical error path and candidate overlays
figure('Position',[1000 1000 1000 1000]);
hold on
plot(N_tosses,mean_abs_error,'.', ...
    'Color',[0.000 0.000 0.000], ...
    'DisplayName','Simulated average absolute error');
plot(N_tosses,root_n_decay_curve,'-', ...
    'Color',[0.000 0.447 0.741], ...
    'DisplayName','Candidate decay: 1/\surd{n}');
```

```
plot(N_tosses,one_over_n_decay_curve,'-', ...
    'Color',[0.301 0.745 0.933], ...
    'DisplayName','Candidate decay: 1/n');
title('Fig 1. Convergence rate of the sample proportion');
ylabel(['Mean absolute deviation of the sample proportion from ' ...
    'the true probability'])
xlabel('Sample size');
legend('show');
hold off
```

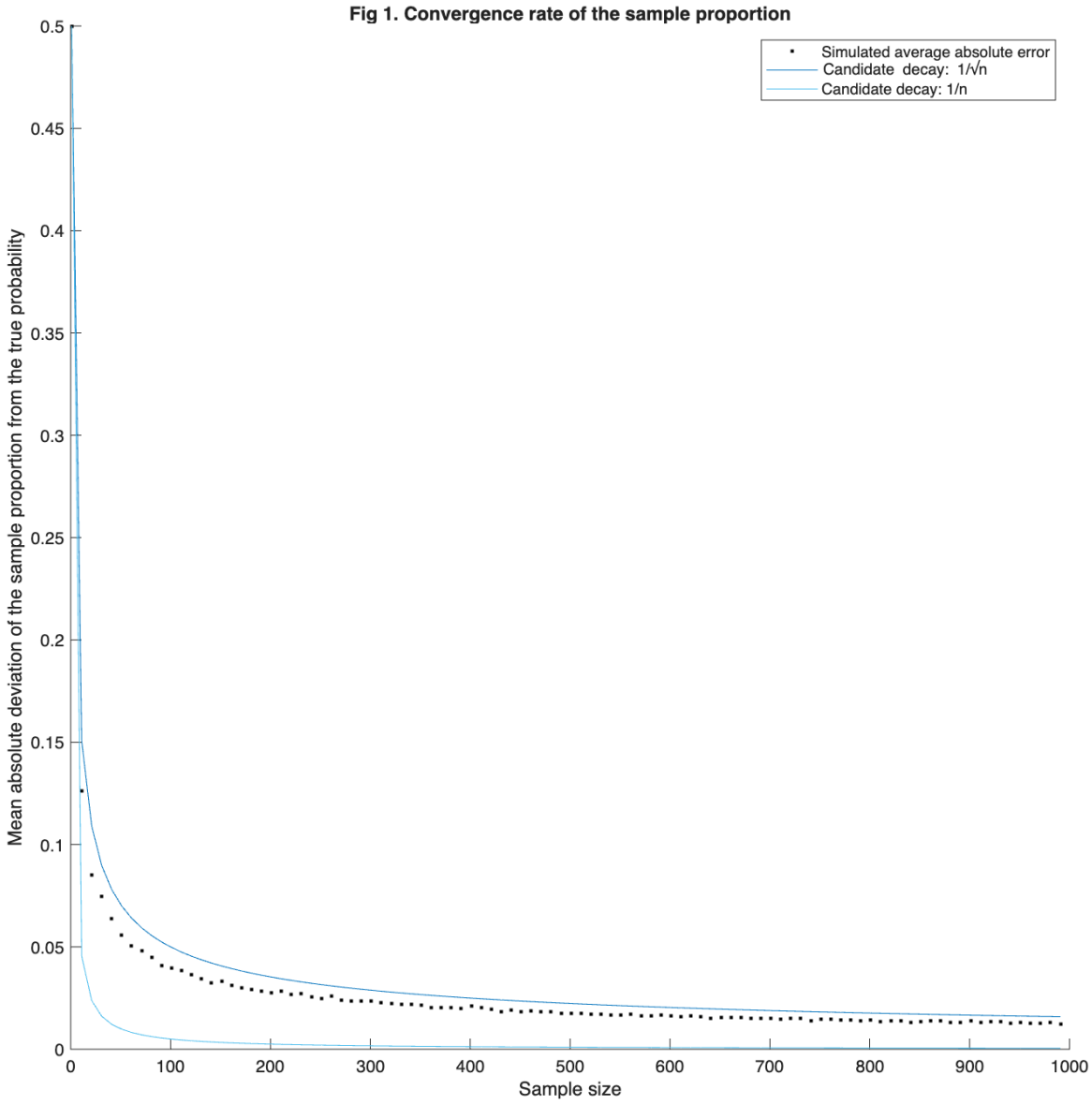


Figure 1: Comparison of the empirical error path with $1/\sqrt{n}$ and $1/n$ decay rates.

4. Convergence in probability and small o_p notation

In Figure 2, we select four representative sample sizes and, for each of them, plot the Monte Carlo realizations of the unscaled estimation error, $\bar{x}_n - \mu$. The figure shows that the dispersion of the unscaled estimation error collapses as the sample size increases. This visual pattern is consistent with the Weak Law of Large Numbers and illustrates convergence in probability using the coin-toss example. That is, according to the Weak Law of Large Numbers, the sample mean $\bar{x}_n$ converges in probability to the population mean μ:

$$\lim_{n\to\infty} \mathbb{P}(|\bar{x}_n - \mu| < \varepsilon) = 1.$$

Equivalently, the probability that the estimation error exceeds any fixed threshold approaches zero as the sample size increases:

$$\lim_{n\to\infty} \mathbb{P}\left(|\bar{x}_n - \mu| \geq \delta\right) = 0.$$

This motivates the meaning of small-o_p notation, namely that the estimation error converges in probability to zero.

A sequence of random variables x_n is said to be of order

$$o_p(a_n)$$

if, for any constant $\delta > 0$,

$$\lim_{n\to\infty} \mathbb{P}\left(\left|\frac{x_n}{a_n}\right| \geq \delta\right) = 0.$$

Applying the definition with

$$x_n = \bar{x}_n - \mu$$

and

$$a_n = 1,$$

gives

$$\lim_{n\to\infty} \mathbb{P}\left(\left|\frac{\bar{x}_n - \mu}{1}\right| \geq \delta\right) = 0.$$

We express this as

$$\bar{x}_n - \mu = o_p(1).$$

The notation $o_p(1)$ indicates that the estimation error converges in probability to zero.

```
%% 7. Visualizing convergence in probability: op(1)

% Plot the unscaled estimation errors
figure('Position',[1000 1000 1000 1000]);
hold on
for j = 1:length(N_tosses)
    plot_n = N_tosses(j);
    % Compute the unscaled estimation error
    unscaled_error_col = means_samples(:,j) - mean_population;
    % Replicate the sample size value for all Monte Carlo simulations
    x_axis_values = repmat(plot_n,N_sim,1);
    % Plot the unscaled estimation errors for this sample size
```

```
    scatter(x_axis_values,unscaled_error_col,5,'.', ...
        'MarkerEdgeColor',[0.000 0.000 0.000], ...
        'MarkerEdgeAlpha',0.25, ...
        'HandleVisibility','off');
end
% Add a reference line at zero estimation error
yline(0,'-', ...
    'Color',[0.000 0.000 0.000], ...
    'HandleVisibility','off');
title('Fig 2. Convergence in probability of the unscaled error');
ylabel(['Unscaled deviation of the sample proportion from the true ' ...
    'probability']);
xlabel('Sample size');
hold off
```

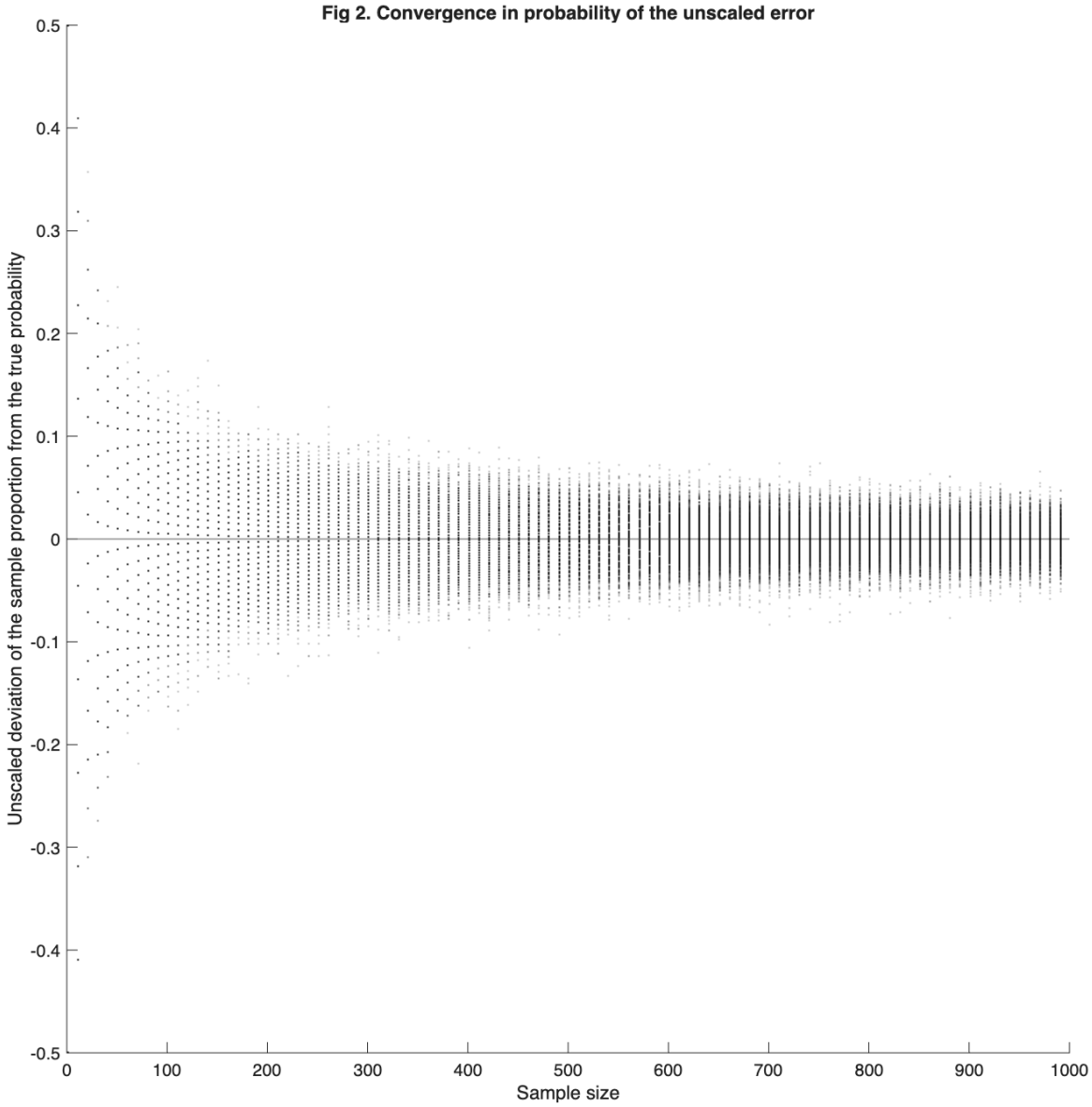


Figure 2: Illustration of convergence in probability.

5. Stochastic boundedness

Figure 2 showed that the unscaled estimation error, $\bar{x}_n - \mu$, becomes progressively smaller as the sample size increases. The next question is whether we can describe more precisely how rapidly this convergence occurs. Figure 1 suggested that the estimation error shrinks at approximately the rate $1/\sqrt{n}$. To investigate whether this is indeed the correct rate of convergence, we multiply the estimation error by $\sqrt{n}$ and consider

$$\sqrt{n}\,(\bar{x}_n - \mu)\,.$$

If the estimation error decreases approximately in proportion to $1/\sqrt{n}$, then multiplying by $\sqrt{n}$ should produce a quantity whose dispersion remains roughly constant as the sample size increases.

Using the same representative sample sizes as in Figure 2, we compute the root-n scaled estimation error,

$$\sqrt{n}\,(\bar{x}_n - \mu)\,,$$

for each Monte Carlo replication and plot the resulting observations against the corresponding sample sizes in Figure 3.

Unlike in Figure 2, the vertical spread of the scaled errors does not collapse toward zero. Instead, the cloud of observations stabilizes within a roughly constant horizontal band. As the sample size increases, the observations neither concentrate at a single point nor spread out toward infinity.

```
%% 8. Visualizing stochastic boundedness: Op(1)

% Plot the root-n scaled estimation errors
figure('Position',[1000 1000 1000 1000]);
hold on
for j = 1:length(N_tosses)
    plot_n = N_tosses(j);
    % Compute the root-n scaled estimation error
    scaled_error_col = sqrt(plot_n) * ...
        (means_samples(:,j) - mean_population);
    % Replicate the sample size value for all Monte Carlo simulations
    x_axis_values = repmat(plot_n,N_sim,1);
    % Plot the root-n scaled estimation errors for this sample size
    scatter(x_axis_values,scaled_error_col,5,'.', ...
    'MarkerEdgeColor',[0.000 0.000 0.000], ...
    'MarkerEdgeAlpha',0.25);
end
% Reference line corresponding to zero estimation error
yline(0,'-', ...
    'Color',[0.850 0.325 0.098]);
title('Fig 3. Stochastic boundedness of the root-n scaled error');
ylabel(['Root-n scaled deviation of the sample proportion from ' ...
    'the true probability'])
xlabel('Sample size');
hold off
```

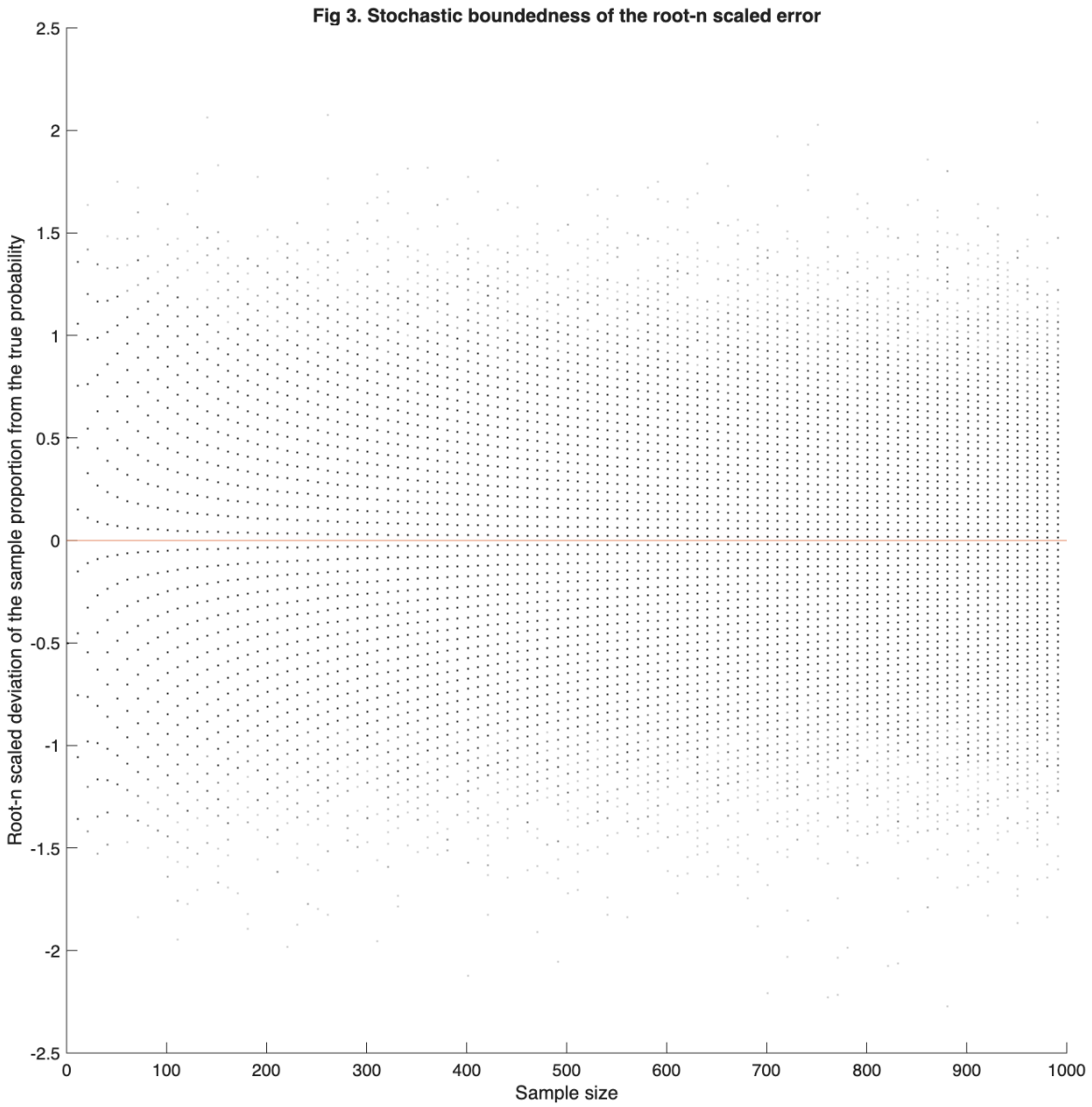


Figure 3: Stochastic boundedness of the root-n-scaled estimation error.

6. Central limit theorem and the Big $\mathcal{O}_p$ notation

Figure 3 showed that the root-n scaled estimation error,

$$\sqrt{n}(\bar{x}_n - \mu),$$

remains confined within a roughly constant band as the sample size increases. This visual pattern suggests that the scaled error does not diverge as n grows. The CLT provides the theoretical explanation for this behavior. According to the CLT, under the assumption that the observations are i.i.d. with mean μ and finite variance σ^2,

$$\bar{x}_n \overset{a}{\sim} \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right),$$

or equivalently,

$$\sqrt{n}(\bar{x}_n - \mu) \xrightarrow{d} \mathcal{N}\left(0, \sigma^2\right).$$

The theorem shows that, the variance of the sample mean is

$$\mathrm{Var}(\bar{x}_n) = \frac{\sigma^2}{n},$$

which shrinks toward zero as the sample size increases, but after multiplying the estimation error by $\sqrt{n}$, the resulting quantity converges to a non-degenerate probability distribution with finite variance. This explains why the cloud of observations in Figure 3 remains within a roughly constant horizontal band as the sample size increases.

This motivates the meaning of the Big-$\mathcal{O}_p$ notation, which formalizes the notion of stochastic boundedness of a random variable.

A sequence of random variables x_n is said to be of order

$$\mathcal{O}_p\left(a_n\right)$$

if, for any arbitrarily small probability $\varepsilon > 0$, there exists a finite constant M and an integer N such that

$$\mathbb{P}\left(\left|\frac{x_n}{a_n}\right| > M\right) < \varepsilon, \quad \text{for all } n > N.$$

This condition states that the sequence, when scaled by a_n, remains bounded in probability.

Applying the definition with

$$x_n = \bar{x}_n - \mu$$

and

$$a_n = \frac{1}{\sqrt{n}},$$

gives

$$\mathbb{P}\left(\left|\sqrt{n}\left(\bar{x}_n - \mu\right)\right| > M\right) < \varepsilon,$$

for sufficiently large n. We express this as

$$\bar{x}_n - \mu = \mathcal{O}_p\left(\frac{1}{\sqrt{n}}\right).$$

This expression states that the estimation error stays within a boundary that shrinks at a rate of $1/\sqrt{n}$, except with a negligibly small probability. Figure 1 suggested that the estimation error shrinks approximately in proportion to $1/\sqrt{n}$.

Equivalently,

$$\sqrt{n}(\bar{x}_n - \mu) = \mathcal{O}_p(1).$$

This states that the root-n-scaled estimation error is stochastically bounded within a fixed range, showing no tendency to grow or shrink as n increases. Figure 3 illustrates this property by showing that the distribution of the scaled estimation error remains of approximately constant magnitude as the sample size increases.

To visualize the CLT implication directly, we estimate the empirical probability density of the root-n scaled error for the largest simulated sample size of 991, and compare it with the theoretical normal density implied by the theorem. The result is shown in Figure 4.

```
%% 9. Visualizing asymptotic normality of the root-n scaled error

% 9.1. Select the largest sample size for the CLT demonstration
n_large = N_tosses(N_samples);

% 9.2. Calculate the root-n scaled estimation error
scaled_error_clt = sqrt(n_large) * ...
    (means_samples(:,N_samples) - mean_population);

% 9.3. Estimate the empirical density of the scaled errors
[f_empirical,x_empirical] = ksdensity(scaled_error_clt);

% 9.4. Calculate the theoretical standard deviation implied by the CLT
std_dev = sqrt(var_population);

% 9.5. Create an evaluation grid for the theoretical Normal density
x_theoretical = linspace(min(x_empirical),max(x_empirical),500);

% 9.6. Evaluate the theoretical Normal density on the grid
f_theoretical = normpdf(x_theoretical,0,std_dev);

% 9.7. Compare the empirical and theoretical densities
figure('Position',[1000 1000 1000 1000]);
hold on
% Plot the empirical kernel density estimate
plot(x_empirical,f_empirical,'-', ...
    'Color',[0.000 0.000 0.000], ...
    'DisplayName','Empirical density of the scaled error');
% Overlay the theoretical Normal density
plot(x_theoretical,f_theoretical,'-', ...
    'Color',[0.850 0.325 0.098], ...
    'DisplayName','Theoretical Normal density');
xlim([-2.5,2.5]);
title('Fig 4. Asymptotic normality of the root-n scaled error');
ylabel('Probability density');
xlabel(['Root-n scaled deviation of the sample proportion from ' ...
    'the true probability']);
```

```
173 legend('show');
174 hold off
```

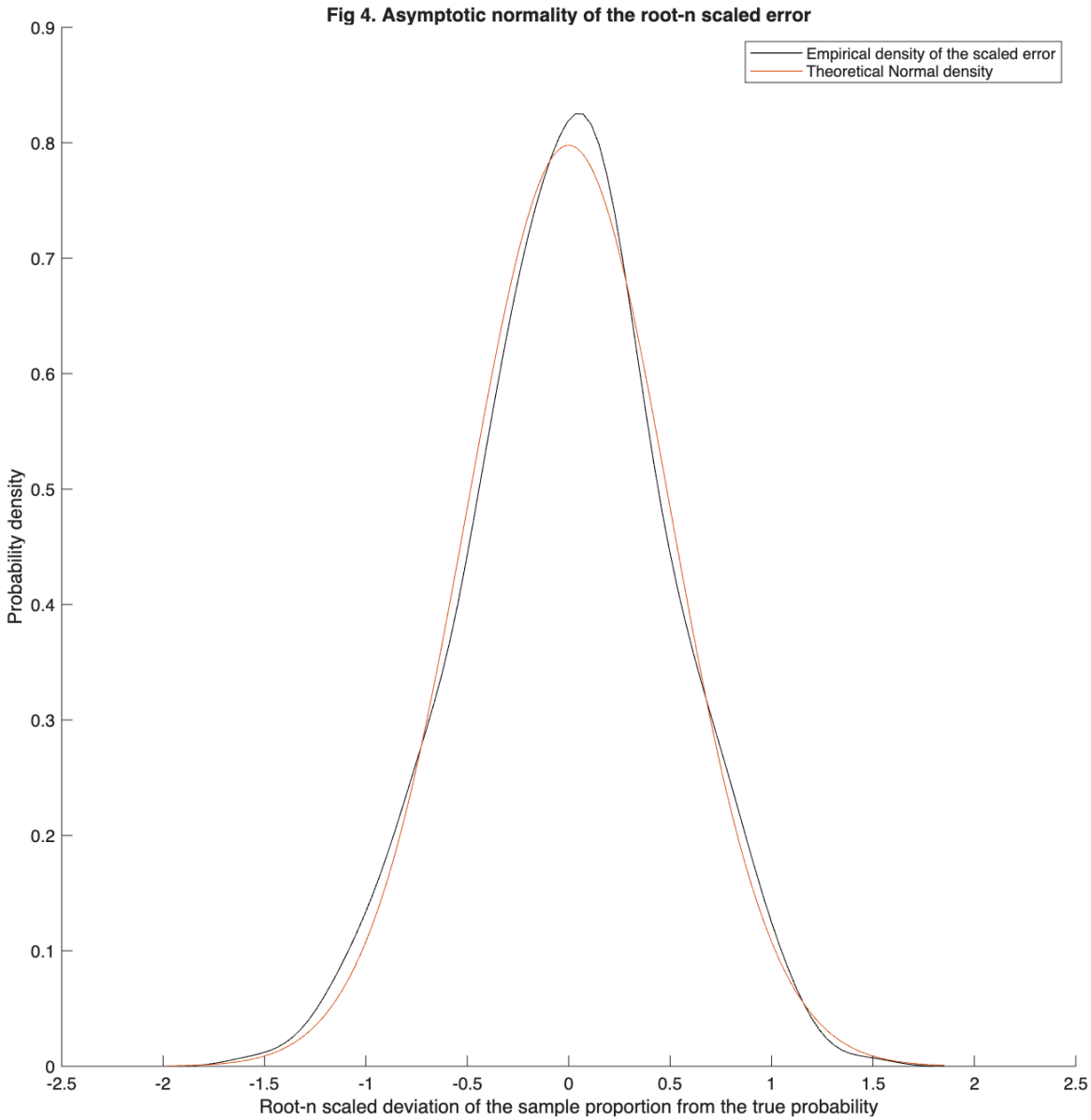


Figure 4: Comparison of the empirical density and the limiting Normal density.

7. Final notes

This file is prepared and copyrighted by Robin van der Meulen and Tunga Kantarcı. This file and the accompanying MATLAB file are available on GitHub and can be accessed using this link.